

Communication 4 — CRC Error Detection

Tutorial Group: Communication

Goal

Extend the IR communication protocol from *Communication 38kHz_Modulated_IR* by adding error detection with a Cyclic Redundancy Check (CRC). The Arduino will calculate a CRC for each message and optionally transmit it with the data. The ESP32 will calculate its own CRC from the received data and compare the result with the transmitted CRC. If the values match, the message can be treated as valid; if they do not match, the ESP32 reports a CRC error.

What You Need

- Arduino Mega 2560 and USB cable
- ESP32-WROOM-32 and USB cable
- Keyestudio IR transmitter and IR receiver modules
- Working *Communication 38kHz_Modulated_IR* system

Why Do We Need a CRC?

A communication receiver can receive a message that looks reasonable even when one or more characters have changed. A CRC provides a way for the receiver to check whether the received data is consistent with the data that was transmitted.

CRC stands for: **Cyclic Redundancy Check**

There is not one single CRC algorithm, as CRCs are a family of error-detection methods. Different variants use different parameters, so both ends of a communication link must use the same variant.

The algorithm used in this tutorial is commonly called: **CRC-16/Modbus**. CRC-16/Modbus is associated with the Modbus industrial communication protocol. You can search for “CRC-16/Modbus” to learn more about the algorithm and compare implementations. For this tutorial, you do not need to derive the CRC mathematics as the CRC calculation function is provided for you.

Part 1 — The CRC Calculation

Both the Arduino and ESP32 will need to use the same CRC function. Copy the function exactly; changing the algorithm on one side will cause the CRC values to disagree.

```
uint16_t calculateCRC(const char *data)
{
    uint16_t crc = 0xFFFF;
    while (*data)
    {
        crc ^= (uint8_t)*data++;
        for (int i = 0; i < 8; i++)
        {
            if (crc & 1)
                crc = (crc >> 1) ^ 0xA001;
            else
                crc >>= 1;
        }
    }
    return crc;
}
```

At a high level, the function processes every character in the message and produces a 16-bit result. The important point for this tutorial is not to memorize the algorithm, but to understand that the same input will always produce the same CRC.

Part 2 — Calculate a CRC for a Known Message

Create a short test program or add temporary code to the Arduino program from *Communication 38kHz_Modulated_IR*. Calculate the CRC for a known message and print the result to the Serial Monitor.

For example:

```
const char *message = "HELLO";
uint16_t crc = calculateCRC(message);
Serial.println(crc, HEX);
```

The ESP32 should produce the same CRC when it calculates the CRC for the same string using the same function. Run it a couple of times to verify, and then change the message. The CRC should change with the new message, but it should always produce the same result for the same input message.

Part 3 — Build and Transmit a JSON Message with CRC

In the previous tutorial, the Arduino transmitted sensor data as a text message. In this tutorial, the data will be represented as a JSON object, and an optional CRC can be appended to the message. We can transmit both the JSON string and the CRC by using a unique separator to separate the JSON string from the CRC. For example, if we use the string constant “\$\$\$” as the separator, the JSON string and CRC can be transmitted as:

```
JSON$$$CRC
```

In the example below, a pre-encoded JSON string is used. The CRC is calculated and both the JSON and CRC values are sent with the separator between the two fields:

```
const char CRC_SEPARATOR[] = "$$$";

const char *jsonEncoded = "{\"v\":4.87,\"temp\":24.7}";
uint16_t crc = calculateCRC(jsonEncoded);

Serial.print(jsonEncoded);
Serial.print(CRC_SEPARATOR);
Serial.println(crc, HEX);
```

Which will result in the following string being transmitted:

```
{"v":4.87,"temp":24.7}$$$F2C1
```

The “\$\$\$” characters separate the JSON data from the optional CRC. The separator is not part of the JSON data and is not included when calculating the CRC.

A message may also be transmitted without a CRC:

```
{"v":4.87,"temp":24.7}
```

This gives the protocol the flexibility to carry either:

```
JSON
```

or:

```
JSON$$$CRC
```

The receiver can determine whether a CRC is present by looking for the “\$\$\$” separator.

Preparing the Received Message

When a CRC is present, the receiver can locate the separator and terminate the JSON string at that point. The following code performs those steps:

```
char *separator = strstr(received, CRC_SEPARATOR);

// If the separator was found, separate it from the CRC, calculate and compare the CRC,
// and return if an error is found
if (separator != NULL)
{
    *separator = '\0';

    uint16_t receivedCRC = strtoul(separator + strlen(CRC_SEPARATOR), NULL, 16);

    uint16_t calculatedCRC = calculateCRC(received);

    if (receivedCRC != calculatedCRC)
    {
        // CRC ERROR: the CRC is invalid so reject the JSON and abort further processing
        //      Do whatever processing you need here to handle the invalid CRC
        return;
    }
}

// Parse the JSON - either the CRC is valid, or no CRC was included
```

The important sequence is to look for the optional separator and, if found, separate the JSON from the CRC, then decode the received CRC, calculate the CRC on the received JSON, and then verify that the two values match.

Important: The CRC is calculated from the exact characters in the JSON string. Braces, quotation marks, field names, punctuation, numeric values, whitespace, and field order all affect the CRC. Two JSON strings can contain the same information but produce different CRC values if their text representation is different.

Part 4 — ESP32: Verify the CRC

Modify the ESP32 program so that it receives the complete ASCII message. If a CRC is present, separate it from the JSON, calculate a new CRC from the received JSON, and compare the two values.

If the CRC values match, parse the JSON and display the sensor values on the OLED. If they do not match, display a clear error message:

```
CRC ERROR
```

When a CRC is present, the receiver should not treat the JSON as valid until the CRC check succeeds.

Suppose the Arduino transmitted the following message:

```
{"v":4.87,"temp":24.7}$$$F2C1
```

If the data is changed during transmission, the received message might instead contain:

```
{"v":4.88,"temp":24.7}$$$F2C1
```

The ESP32 calculates the CRC from the received JSON. Because the JSON changed, the calculated CRC will no longer match the transmitted CRC. The ESP32 therefore reports a CRC error rather than treating the changed value as valid.

Notice that the changed value may still look completely reasonable to a person. The CRC provides an independent check of the data.

Part 5 — Add CRC Checking to the Communication Program

The communication program developed in *Communication 38kHz_Modulated_IR* can be extended to use the CRC technique developed in this tutorial.

Experiments

Try the following experiments:

- Test the transmission both with and without the optional CRC and verify it works
- Modify the transmission so that only the separator is included, but the CRC value is missing. How should your code handle this case?
- Add extra characters after the valid JSON and CRC. What does the receiver do? Does your current parsing code detect that the entire message is not in the expected format?
- Make a slight modification to the CRC algorithm on either the sending or receiving side.
- Use two different, but valid CRC algorithms, on the sending and receiving side.
- Modify the Arduino program temporarily so that the transmitted JSON is changed after the CRC is calculated, or otherwise create a controlled mismatch between the data and its CRC.
- Modify the server program in *ESP32 HTTP_POST_Python_Server* to look for and process the optional CRC value and flag any invalid CRCs as an error.
- Modify the formatting of the JSON without changing the information it contains. For example, change the field order or add/remove whitespace. What happens to the CRC, and why?
- Investigate extending Part 5 so that any invalid messages send an error back to the sender to ask for a retransmission. How could you handle this?
- If you know that all senders have been upgraded to always transmit a CRC, consider modifying the receiver code to expect the CRC and report the missing CRC as an error.

What You Should Understand

- CRC means Cyclic Redundancy Check.
- CRC is an error-detection technique, not an encryption method.
- CRC-16/Modbus is one specific CRC variant.
- Both sides of a communication link must use the same CRC definition.
- A CRC can detect many accidental changes to transmitted data.
- When a CRC is present, the receiver should not treat received data as valid until the CRC check succeeds.
- A message can look reasonable to a human and still fail its CRC check.
- A CRC can be included as an optional part of a JSON-based message.

Before Moving On

- ☐ I can explain what a CRC is used for.
- ☐ I know that CRC has multiple variants.
- ☐ I can identify CRC-16/Modbus as the algorithm used in this tutorial.
- ☐ I can explain why the same CRC calculation is required on both sides.
- ☐ I can explain the JSON\$\$\$CRC message structure.
- ☐ I can demonstrate a CRC error by intentionally changing data.
- ☐ I can explain why the ESP32 should reject a message when the CRC does not match.
- ☐ I can explain how a CRC can be used to determine whether a received message contains an error.